

As an example, $P = [60, 160, 90, 130, 95, 160]$. But here comes the first challenge in AI and machine learning. What if there are a billion health records. The task will be incomplete even if tens of thousands of professionals work to manually extract data from these billion health records. Hence, AI and machine learning applications try to extract data from these records using programs.

The second challenge in AI and machine learning is data interpretation. This is a vast field and there are a number of techniques worth exploring. We will go through the most relevant of them in the coming articles in this series. AI and machine learning based applications also face hardware challenges, in addition to mathematical/computational challenges. With the huge amount of data being processed, data storage, processor speed, power consumption, etc, also become a major challenge for AI based applications. Challenges apart, I believe it is time for us to write the first line of AI code.

We will write a simple Python script to add two vectors. For that, we use a Python library called NumPy. This is a Python library that supports multi-dimensional matrices (arrays) and a lot of mathematical functions to operate on them. The command 'pip3 install numpy' will install the package NumPy for Python 3. Notice that NumPy would have been preinstalled if you were using JupyterLab, Google Colab or Anaconda. However, we will operate from the Linux terminal for the first few articles in this series on AI for ease of use. Execute the command 'python3' on the Linux terminal to work from the Python console. This console is software that allows line-by-line execution of Python code. Figure 3 shows the line-by-line execution of the Python code to add two vectors and show the output on the terminal.

First, let us try to understand the code. An important note before we proceed any further. Since this tutorial assumes very little programming experience, I will label lines of

```
>>> import numpy as np
>>> a = np.array([11, 22, 33])
>>> b = np.array([44, 55, 66])
>>> c = np.add(a, b)
>>> print(c)
[55 77 99]
```

Figure 3: Python code to find the sum of two row vectors

code as either (*basic*) or (*AI*). Lines labelled (*basic*) are part of classical Python code, whereas lines labelled (*AI*) are part of the Python code for developing AI applications. I know such a classification is not necessary or very meaningful. However, I want to distinguish between basic Python and advanced Python so that programmers with both basic and intermediate skills in programming will find this tutorial useful.

The line of code *import numpy as np* (*basic*) imports the library NumPy and names it as *np*. The *import* statement in Python is similar to the *#include* statement of C/C++ to use header files and the *import* statement of Java to use packages. The lines of code *a = np.array([11, 22, 33])* and *b = np.array([44, 55, 66])* (*AI*) create two one-dimensional arrays named 'a' and 'b'. Let me do one more simplification for the sake of understanding. For the time being, assume a vector is equivalent to a one-dimensional array. The line of code *c = np.add(a, b)* (*AI*) adds the two vectors named 'a' and 'b' and stores the result in a vector named 'c'. Of course, naming variables as 'a', 'b', 'c', etc, is a bad programming practice, but mathematicians tend to differ and name vectors as 'u', 'v', 'w', etc. If you are an absolute beginner in Python programming, please learn how variables in Python work.

Finally, the line of code *print(c)* (*basic*) prints the value of the object, the vector [55 77 99], on the

terminal. This is a line of basic Python code. The vector $c = [55=11+44, 77=22+55, 99=33+66]$ gives you a hint about vector and matrix addition. However, if you want to formally learn how vectors and matrices are added, and if you don't have any good mathematical textbooks on the topic available at your disposal, I suggest you go through the Wikipedia article on matrix addition. A quick search on the Internet will show you that a classic C, C++ or Java program to add two vectors takes a lot more of code. This itself shows how suitable Python is for handling vectors and matrices, the lifeline of linear algebra. The strength of Python will be further appreciated as we perform more and more complex vector operations.

Before we conclude this article, I want to make two disclaimers. First, the programming example just discussed above deals with the addition of two row vectors (1×3 matrices to be precise). But a real machine learning application might be dealing with a 1000000×1000000 matrix. However, with practice and patience we will be able to handle these. The second disclaimer is that many of the definitions given in this article involve gross simplifications and some hand-waving. But, as mentioned earlier, I will give formal definitions for any term I have defined loosely before the conclusion of this series.

Now it is time for us to wind up this article. I request all of you to install the necessary software mentioned in this article and run each and every line of code discussed here. We will begin the next article in this series by discussing the history, scope and future of AI, and then proceed deeper into matrix theory, the backbone of linear algebra. **END** 🐧

 By: Deepu Benson

The author is a free software enthusiast. His area of interest is theoretical computer science. The open source tools of his choice include ns-2 and ns-3. The author maintains his own technical blog too.